Embedded Linux application development



© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!



Contents

- Application development
  - Developing applications on embedded Linux
  - Building your applications
- Debugging and analysis tools
  - Debuggers
  - Remote debugging
  - Tracing and profiling



Developing applications on embedded Linux



Application development

- An embedded Linux system is just a normal Linux system, with usually a smaller selection of components
- In terms of application development, developing on embedded Linux is exactly the same as developing on a desktop Linux system
- All existing skills can be reused, without any particular adaptation
- All existing libraries, either third-party or in-house, can be integrated into the embedded Linux system
  - Taking into account, of course, the limitation of the embedded systems in terms of performance, storage and memory
- Application development could start on x86, even before the hardware is available.



Leverage existing libraries and languages

- Many developers getting started with embedded Linux limit themselves to C, sometimes C++, and the C/C++ standard library.
- However, there are a lot of libraries and languages that can help you accelerate and simplify your application development
  - Compiled languages like Rust and Go are increasingly popular
  - Interpreted languages, especially Python
  - Higher-level libraries: Qt, Glib, Boost, and many more
- Make sure to evaluate what is the right choice for your project, but pay attention to
  - Footprint and performance on low-end platforms
  - Use well-maintained and well-known technologies



Building your applications/libraries

- Even for simple applications or libraries, make use of a build system
  - CMake
  - Meson
- This will simplify
  - the build process of your application
  - the life of developers joining your project
  - the packaging of your application into an embedded Linux build system



Getting started with *meson*

Minimal `meson.build`

```
project('example', 'c')
executable('demo', 'main.c')
```

`meson.build` for multiple programs and source files

```
project('example', 'c')
src_demo1 = ['demo1.c', 'foo1.c']
executable('demo1', src_demo1)
src_demo2 = ['demo2.c', 'foo2.c']
executable('demo2', src_demo2)
```



Options with *meson*

meson_options.txt

```
option('demo-debug', type : 'feature', value : 'disabled')
```

meson.build

```
project('tutorial', 'c')
demo_c_args = []
if get_option('demo-debug').enabled()
   demo_c_args += '-DDEBUG'
endif
executable('demo', 'main.c', c_args: demo_c_args)
```



Library dependencies with *meson*

`meson.build`

```
project('tutorial', 'c')
gtkdep = dependency('gtk+-3.0')
executable('demo', 'main.c', dependencies : gtkdep)
```

The dependency `gtk+-3.0` is searched using `pkg-config`.

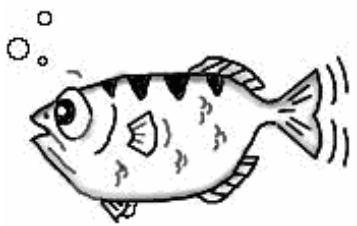


Debugging



GDB: GNU Project Debugger

- The debugger on GNU/Linux, available for most embedded architectures.
- Supported languages: C, C++, Pascal, Objective-C, Fortran, Ada...
- Command-line interface
- Integration in many graphical IDEs
- Can be used to
  - control the execution of a running program, set breakpoints or change internal variables
  - to see what a program was doing when it crashed: post mortem analysis
- https://www.gnu.org/software/gdb/
- https://en.wikipedia.org/wiki/Gdb
- New alternative: *lldb* (https://lldb.llvm.org/) from the LLVM project.





GDB crash course (1/3)

- GDB is used mainly to debug a process by starting it with *gdb*
  - `$ gdb <program>`
- GDB can also be attached to running processes using the program PID
  - `$ gdb -p <pid>`
- When using GDB to start a program, the program needs to be run with
  - `(gdb) run [prog_arg1 [prog_arg2] ...]`



GDB crash course (2/3)

A few useful GDB commands

- `break foobar` (`b`)
  Put a breakpoint at the entry of function `foobar()`
- `break foobar.c:42`
  Put a breakpoint in `foobar.c`, line 42
- `print var`, `print $reg` or `print task->files[0].fd` (`p`)
  Print the variable `var`, the register `$reg` or a more complicated reference. GDB can also nicely display structures with all their members
- `info registers`
  Display architecture registers



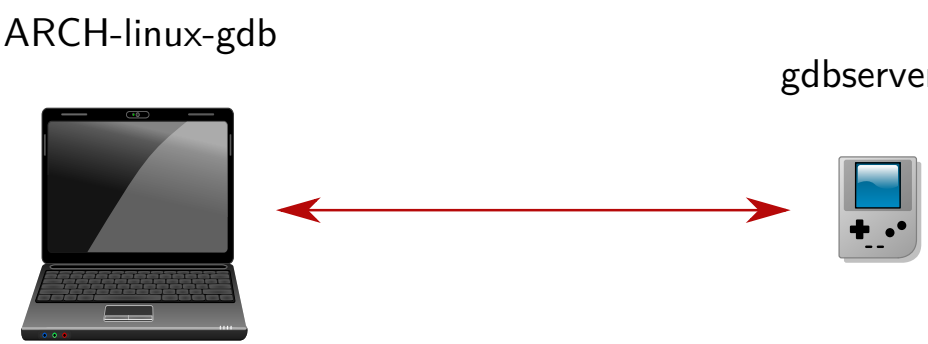
GDB crash course (3/3)

- `continue` (`c`)
  Continue the execution after a breakpoint
- `next` (`n`)
  Continue to the next line, stepping over function calls
- `step` (`s`)
  Continue to the next line, entering into subfunctions
- `stepi` (`si`)
  Continue to the next instruction
- `finish`
  Execute up to function return

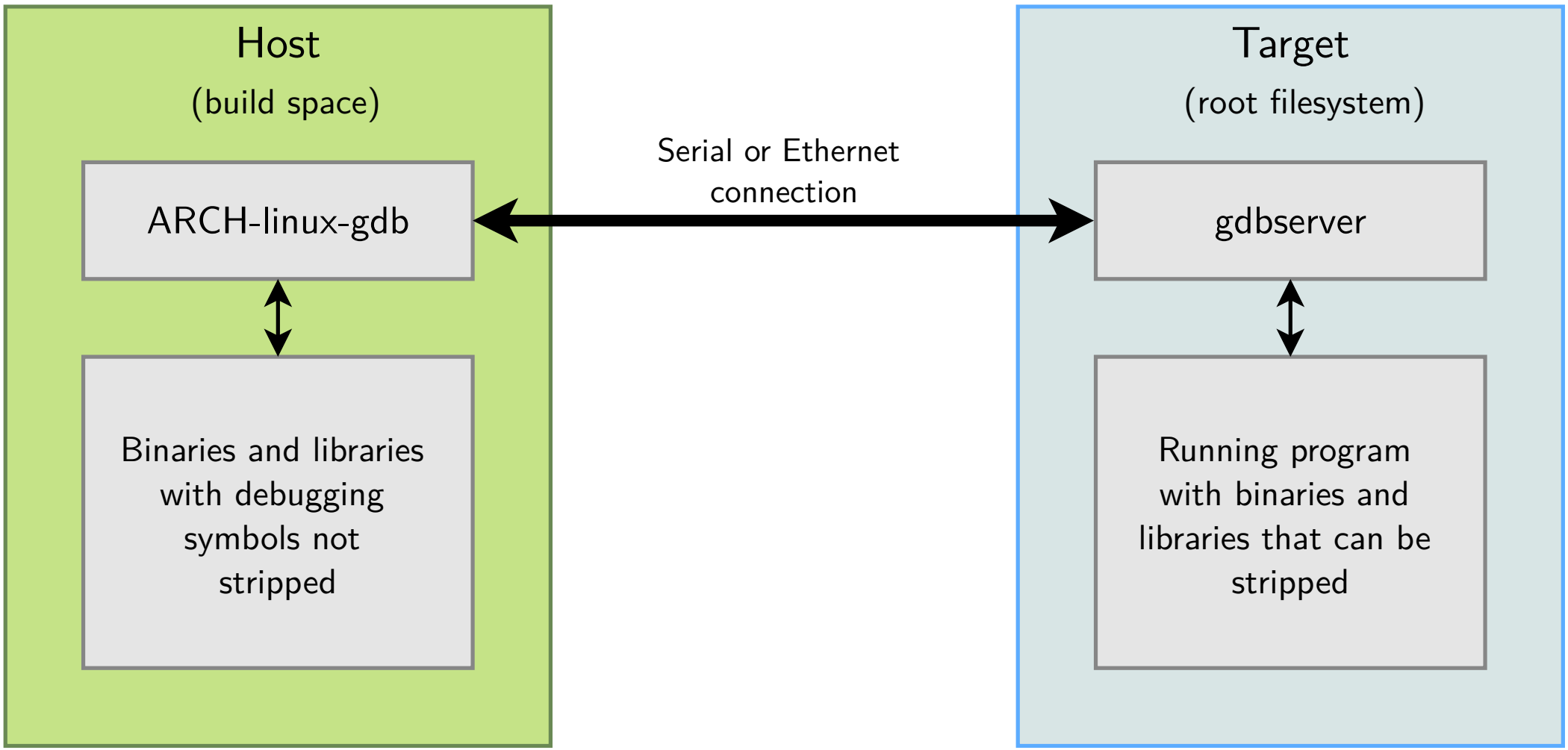


Remote debugging

- In a non-embedded environment, debugging takes place using `gdb` or one of its front-ends.
- `gdb` has direct access to the binary and libraries compiled with debugging symbols, which is often false for embedded systems (binaries are stripped, without debug_info) to save storage space.
- For the same reason, embedding the `gdb` program on embedded targets is rarely desirable (2.4 MB on x86).
- Remote debugging is preferred
  - `ARCH-linux-gdb` is used on the development workstation, offering all its features.
  - `gdbserver` is used on the target system (only 400 KB on arm).



Remote debugging: architecture





Remote debugging: target setup

- On the target, run a program through `gdbserver`.
  Program execution will not start immediately.
  `gdbserver :<port> <executable> <args>`
  `gdbserver /dev/ttyS0 <executable> <args>`
- Otherwise, attach `gdbserver` to an already running program:
  `gdbserver --attach :<port> <pid>`
- You can also start gdbserver without passing any program to start or attach (and set the target program later, on client side):
  `gdbserver --multi :<port>`



Remote debugging: host setup

- Then, on the host, start `ARCH-linux-gdb <executable>`, and use the following `gdb` commands:
  - To tell `gdb` where shared libraries are:
    `gdb> set sysroot <library-path>` (typically path to build space without `lib/`)
  - To connect to the target:
    `gdb> target remote <ip-addr>:<port>` (networking)
    `gdb> target remote /dev/ttyUSB0` (serial link)
    - Make sure to replace `target remote` with `target extended-remote` if you have started gdbserver with the `--multi` option
  - If you did not set the program to debug on gdbserver commandline:
    `gdb> set remote exec-file <path_to_program_on_target>`



Coredumps for post mortem analysis

- It is sometimes not possible to have a debugger attached when a crash occurs
- Fortunately, Linux can generate a `core` file (a snapshot of the whole process memory at the moment of the crash), in the ELF format. gdb can use this `core` file to let us analyze the state of the crashed application
- On the target
  - Use `ulimit -c unlimited` in the shell starting the application, to enable the generation of a `core` file when a crash occurs
  - The output name and path for the coredump file can be modified using `/proc/sys/kernel/core_pattern` (see man 5 core)
    - Example: `echo /tmp/mycore > /proc/sys/kernel/core_pattern`
  - Depending on the system configuration, the `core_pattern` file may be rewritten automatically by some software to handle core files or even disable core generation (eg: systemd)
- On the host
  - After the crash, transfer the `core` file from the target to the host, and run `ARCH-linux-gdb application-binary core-file`



minicoredumper

- Coredumps can be huge for complex applications
- minicoredumper is a userspace tool based on the standard core dump feature
  - Based on the possibility to redirect the core dump output to a user space program via a pipe
- Based on a JSON configuration file, it can:
  - save only the relevant sections (stack, heap, selected ELF sections)
  - compress the output file
  - save additional information from `/proc`
- https://github.com/diamon/minicoredumper
- "Efficient and Practical Capturing of Crash Data on Embedded Systems"
  - Presentation by minicoredumper author John Ogness
  - Video: https://www.youtube.com/watch?v=q2zmwrgLJGs
  - Slides: elinux.org/images/8/81/Eoss2023_ogness_minicoredumper.pdf